

Predicting Unusual Household and Housing Profiles Related to Social Tenancy Risk Using Machine Learning

Ahmed Zaus Zahid, Afnan Ali, Aminath Iuzaaz Ismail

Loading the data

The AHS dataset is a massive dataset. However, most of the data is not related to our specific usecase and we will not be needing them. Before we can train our models on the data, we need to identify which columns we need and treat the data which requires cleaning.

In []: `import pandas as pd`

```
# the columns we need from the dataset
```

```
ahs_cols = [
    "CONTROL", "TENURE", "WEIGHT",
    "HUDSUB", "RENTSUB", "RENTCNTRL",
    "HINCP", "FINCP", "PERPOVLVL", "FS",
    "NUMPEOPLE", "BEDROOMS", "TOTROOMS",
    "NUMADULTS", "NUMELDERS", "NUMYNGKIDS", "NUMOLDKIDS", "UNITSIZE",
    "RENT", "TOTHCAMT", "UTILAMT",
    "HIAFFORD", "HIBEHINDFRQ", "HIHALF", "HINUMOVE",
    "HIEVICNOTE", "HIEVICHT", "HIEVICLK", "HINOWHERE",
    "MGRONSITE",
    "NUMNONREL", "NUMSECFAM", "NUMSUBFAM",
    "OCCJANUR", "OCCFEBRU", "OCCMARCH", "OCCAPRIL", "OCCMAY", "OCCJUNE",
    "OCCJULY", "OCCAUGUST", "OCCSEPTEM", "OCCOCTOB", "OCCNOVEM", "OCCDECEM",
    "OCCYRRND", "MONLSTOCC", "VACMONTHS",
]
```

```
df = pd.read_csv(
    "household.csv",          # AHS 2023 national PUF file
    quotechar="'",           # strips the single quotes around every value
    usecols=lambda c: c in ahs_cols, # only load these columns
)
```

```
print(df.shape)
print(df.head())
```

```
(55669, 48)
  CONTROL  TOTROOMS  PERPOVLVL  RENT  TENURE  HIEVICLK  HIAFFORD  HIHALF  \
0  11000002      6      -6  1600    -6      -6      -6      -6
1  11000003      4      199   840     2       4       4       2
2  11000005      7      501   -6     1      -6      -6      -6
3  11000006      5      232   -6     1      -6      -6      -6
4  11000008      3      231   800     2       2       2       1

  OCCJANUR  OCCFEBRU  ...  HIBEHINDFRQ  HINOWHERE  HIEVICHT  HIEVICNOTE  \
0         1         1  ...           -6           -6           -6           -6
1        -6        -6  ...            6            0            2           -6
2        -6        -6  ...           -6            0           -6           -6
3        -6        -6  ...           -6           -6           -6           -6
4        -6        -6  ...            2            0            1            1

  HINUMOVE  MGRONSITE  HUDSUB  HINCP  FINCP  TOTHCAMT
0        -6        -6     -6     -6     -6        -6
1         1        -6      3  48000  48000    1093
2         0        -6     -6  292500  292500    810
3        -6        -6     -6  56000  56000    489
4         0         4      3  36000  36000    845
```

```
[5 rows x 48 columns]
```

Running the cell, we can already see that our data needs cleaning even though the printed section is only a small slice of the full data. The AHS dataset uses -6 and -9 to represent Null values.

Analysing the data for cleaning

```
In [28]: r = df[df["TENURE"] == 2].replace([-6, -9], pd.NA) # renters only, codes -> NaN
missing = r.isna().mean().sort_values(ascending=False)
print((missing * 100).round(1))
print("renter rows:", len(r))
```

```

OCCAUGUST    100.0
OCCJULY      100.0
OCCJUNE      100.0
OCCMAY       100.0
OCCAPRIL     100.0
OCCMARCH     100.0
OCCFEBRU     100.0
OCCJANUR     100.0
OCCDECEM     100.0
VACMONTHS    100.0
MONLSTOCC    100.0
OCCYRRND     100.0
OCCOCTOB     100.0
OCCNOVEM     100.0
OCCSEPTE     100.0
HIEVICNOTE   98.1
RENTCNTRL    87.6
HIHALF       52.6
HIEVICK      51.7
HIAFFORD     51.7
HIBEHINDFRQ  51.6
HINOWHERE    51.6
HIEVICTHT    51.5
HINUMOVE     50.7
MGRONSITE    31.0
FS           30.0
UNITSIZE     18.0
RENTSUB      2.0
TENURE       0.0
PERPOVLVL    0.0
TOTROOMS     0.0
CONTROL      0.0
RENT         0.0
NUMSUBFAM    0.0
NUMOLDKIDS   0.0
NUMYNGKIDS   0.0
NUMNONREL    0.0
NUMADULTS    0.0
NUMELDERS    0.0
WEIGHT       0.0
NUMSECFAM    0.0
UTILAMT      0.0
NUMPEOPLE    0.0
BEDROOMS     0.0
HUDSUB       0.0
HINCP        0.0
FINCP        0.0
TOTHCAMT     0.0
dtype: float64
renter rows: 19735

```

Filtering by `TENURE == 2` (rented) and checking for missing data values per column. We have to drop all the columns without enough data. Columns that have 50% missing data can be considered, however we will ignore those for now and come back to it if the model requires more signals.

Updated columns after removing columns with missing data

```

In [29]: KEEP_COLS = [
    "CONTROL", "TENURE", "WEIGHT",
    "HUDSUB", "RENTSUB", "RENTCNTRL",
    "HINCP", "FINCP", "PERPOVLVL", "FS",
    "NUMPEOPLE", "BEDROOMS", "TOTROOMS",
    "NUMADULTS", "NUMELDERS", "NUMYNGKIDS", "NUMOLDKIDS", "UNITSIZE",
    "RENT", "TOTHCAMT", "UTILAMT",
    "MGRONSITE",
    "NUMNONREL", "NUMSECFAM", "NUMSUBFAM",
]

DROP_ALWAYS = (
    ["OCCJANUR", "OCCFEBRU", "OCCMARCH", "OCCAPRIL", "OCCMAY", "OCCJUNE",
     "OCCJULY", "OCCAUGUST", "OCCSEPTE", "OCCOCTOB", "OCCNOVEM", "OCCDECEM",
     "OCCYRRND", "VACMONTHS", "MONLSTOCC"]          # 100% empty on renters
    + ["HIEVICNOTE"]                                # 98% empty (skip pattern)
    + ["HIAFFORD", "HIBEHINDFRQ", "HIHALF", "HINUMOVE",
       "HIEVICK", "HIEVICTHT", "HINOWHERE"]         # ~50% (split-sample)
)

```

Loading the data again

```

In [30]: df = pd.read_csv(
    "household.csv",
    quotechar="'",
    usecols=lambda c: c in KEEP_COLS,
)
print(df.shape)
df.head()

```

(55669, 25)

```
Out[30]:
```

	CONTROL	TOTROOMS	PERPOVLVL	RENT	TENURE	RENTCNTRL	RENTSUB	WEIGHT	NUMELDERS	NUMADULTS	...	NUMPEOPLE	UNITSIZE	BED
0	11000002	6	-6	1600	-6	-6	8	813.890194	-6	-6	...	-6	7	
1	11000003	4	199	840	2	-6	8	581.103231	0	2	...	3	-9	
2	11000005	7	501	-6	1	-6	-6	7335.965001	2	2	...	2	6	
3	11000006	5	232	-6	1	-6	-6	6562.865941	0	2	...	3	4	
4	11000008	3	231	800	2	-6	8	1490.800600	0	1	...	1	-9	

5 rows × 25 columns

Replacing -6 and -9 with NA

```
In [31]: df = df.replace([-6, -9], pd.NA)
df = df[df["TENURE"] == 2].copy()
print(f"{len(df)} renter rows")
```

19735 renter rows

Dropping columns

```
In [32]: df = df.drop(columns=[c for c in DROP_ALWAYS if c in df.columns])
print(f"{df.shape[1]} columns kept")
```

25 columns kept

Recoding NA flags to 0

```
In [33]: for col in ["RENTCNTRL", "MGRONSITE"]:
df[col] = df[col].fillna(0)
```

```
In [34]: for col in df.columns:
df[col] = pd.to_numeric(df[col], errors="coerce")
df.dtypes
```

```
Out[34]: CONTROL      int64
TOTROOMS      int64
PERPOVLVL     int64
RENT          int64
TENURE        int64
RENTCNTRL     int64
RENTSUB       float64
WEIGHT        float64
NUMELDERS     int64
NUMADULTS     int64
NUMNONREL     int64
NUMYNGKIDS    int64
NUMOLDKIDS    int64
NUMSUBFAM     int64
NUMSECFAM     int64
NUMPEOPLE     int64
UNITSIZE      float64
BEDROOMS     int64
UTILAMT      int64
FS           float64
MGRONSITE     int64
HUDSUB        int64
HINCP         int64
FINCP         int64
TOTHCAMT      int64
dtype: object
```

```
In [35]: df["persons_per_room"] = df["NUMPEOPLE"] / df["TOTROOMS"]
df["is_crowded"] = (df["persons_per_room"] > 1).astype(int)
df["is_crowded"].value_counts()
```

```
Out[35]: is_crowded
0      19152
1       583
Name: count, dtype: int64
```

```
In [36]: miss = (df.isna().mean() * 100).round(1).sort_values(ascending=False)
miss[miss > 0]
```

```
Out[36]: FS          30.0
UNITSIZE      18.0
RENTSUB       2.0
dtype: float64
```

```
In [ ]: df = df.drop(columns=["UNITSIZE"])
df = df.drop(columns=["FS"])
```